

PicoLab

Visual AI learning coach for STEM — Code Overview

GitHub Repository: <https://github.com/Alehcs/PicoLab>

Tech stack

- React
- Vite
- TypeScript
- Tailwind CSS
- Express
- Mock backend REST API
- Deterministic diagnostic engine

This document showcases the source developed for PicoLab. Generated files, dependencies, and build artifacts (node_modules, dist, .git, caches, lockfiles, binary media, backup bundles) are intentionally excluded.

Key Architecture Summary

- **Page-based workflow** — each step of the student loop is a route under `src/pages/` wired in `routes.tsx`.
- **Shared component system** — primitives (Button, Card, Badge) plus feature components under `src/components/`.
- **Backend-first API client** — `picoLabApi` calls the Express mock first and falls back to a deterministic local mock on failure/timeout.
- **Mock backend** — Express + TypeScript exposes contract-shaped REST endpoints under `server/src/routes/`.
- **Diagnostic engine** — classifies a step/answer into learning signals (units, formula, graph, algebra, concept, reading).
- **Local persistence** — practice progress, signals, Ask Pico history, and Visual Lab context persist via `localStorage/sessionStorage`.
- **Context hand-off** — Smart Notebook stores a signal-based suggestion the Visual Lab reads to open the matching template.

Repository Structure

PicoLab/	
■■■ package.json	scripts + dependencies
■■■ vite.config.ts	Vite build config
■■■ tailwind.config.js	design tokens / safelist
■■■ tsconfig.json	TypeScript project refs
■■■ index.html	app entry
■■■ README.md	
■■■ docs/	architecture + demo notes
■ ■■■ api-contracts.md	
■ ■■■ demo-script.md	
■ ■■■ diagnostic-engine.md	
■ ■■■ learning-signal-taxonomy.md	
■ ■■■ integration-roadmap.md	
■■■ src/	
■ ■■■ main.tsx	React entry
■ ■■■ app/	App shell + route table

```

■ ■ ■■■ App.tsx
■ ■ ■■■ routes.tsx
■ ■■■ pages/ page-based student workflow
■ ■ ■■■ AddProblemPage.tsx
■ ■ ■■■ ScanConfirmPage.tsx
■ ■ ■■■ SmartNotebookPage.tsx
■ ■ ■■■ VisualLabPage.tsx
■ ■ ■■■ PracticeMissionsPage.tsx
■ ■ ■■■ GrowthMapPage.tsx
■ ■ ■■■ GrowthPathPage.tsx
■ ■ ■■■ ProfilePage.tsx
■ ■ ■■■ SettingsPage.tsx
■ ■■■ components/ shared component system
■ ■ ■■■ ui/ Button, Card, Badge, Tabs, ProgressBar
■ ■ ■■■ layout/ AppShell, Sidebar, PageHeader
■ ■ ■■■ notebook/ Smart Notebook step cards
■ ■ ■■■ practice/ Daily / Focus / Random mission cards
■ ■ ■■■ visual-lab/ motion + graph simulations
■ ■ ■■■ growth/ Growth Map + Roadmap cards
■ ■ ■■■ pico/ mascot + Ask Pico drawer
■ ■■■ services/ app logic + API client
■ ■ ■■■ apiClient.ts fetch wrapper w/ timeout
■ ■ ■■■ picolabApi.ts backend-first + local fallback
■ ■ ■■■ mockPicolabApi.ts deterministic local mock
■ ■ ■■■ diagnosticEngine.ts learning-signal classifier
■ ■ ■■■ practiceProgress.ts localStorage progress
■ ■ ■■■ visualLabSuggestion.ts notebook -> Visual Lab context
■ ■■■ data/ mock content + taxonomy
■ ■ ■■■ learningSignals.ts
■ ■ ■■■ mockMissions.ts
■ ■ ■■■ mockNotebook.ts
■ ■■■ types/ shared TypeScript types
■ ■■■ styles/ design tokens + globals.css
■ ■■■ server/ Express + TypeScript mock API
■ ■■■ src/index.ts server bootstrap
■ ■■■ src/routes/ REST endpoints
■ ■■■ problems.ts notebooks.ts practice.ts
■ ■■■ growth.ts profile.ts askPico.ts
■ ■■■ visualLab.ts settings.ts health.ts
■ ■■■ src/services/diagnosticEngine.ts
■ ■■■ src/data/learningSignals.ts

```

Important Code Sections

Readable excerpts from the most important source files. Long lines are soft-wrapped; each header notes how many lines are shown.

[src/app/routes.tsx \(64 lines\)](#)

```
import type { ReactNode } from 'react';
import {
  BookOpen,
  FlaskConical,
  Home,
  type LucideIcon,
  Map,
  Plus,
  Route,
  Settings,
  Target,
  User,
} from 'lucide-react';
import { AddProblemPage } from '../pages/AddProblemPage';
import { GrowthMapPage } from '../pages/GrowthMapPage';
import { GrowthPathPage } from '../pages/GrowthPathPage';
import { HomePage } from '../pages/HomePage';
import { PracticeMissionsPage } from '../pages/PracticeMissionsPage';
import { ProfilePage } from '../pages/ProfilePage';
import { ScanConfirmPage } from '../pages/ScanConfirmPage';
import { SettingsPage } from '../pages/SettingsPage';
import { SmartNotebookPage } from '../pages/SmartNotebookPage';
import { VisualLabPage } from '../pages/VisualLabPage';

export type AppRoute = {
  path: string;
  label: string;
  icon: LucideIcon;
  element: ReactNode;
  sidebar?: boolean;
  footer?: boolean;
};

export const appRoutes: AppRoute[] = [
  { path: '/', label: 'Home', icon: Home, element: <HomePage /> },
  { path: '/add-problem', label: 'Add Problem', icon: Plus, element: <AddProblemPage /> },
  {
    path: '/scan-confirm',
    label: 'Scan & Confirm',
    icon: Plus,
    element: <ScanConfirmPage />,
    sidebar: false,
  },
  {
    path: '/smart-notebook',
    label: 'Smart Notebook',
    icon: BookOpen,
    element: <SmartNotebookPage />,
  },
  { path: '/visual-lab', label: 'Visual Lab', icon: FlaskConical, element: <VisualLabPage /> },
  { path: '/growth-map', label: 'Growth Map', icon: Map, element: <GrowthMapPage /> },
  { path: '/growth-path', label: 'Roadmap', icon: Route, element: <GrowthPathPage /> },
  {
    path: '/practice-missions',
    label: 'Practice Missions',
    icon: Target,
    element: <PracticeMissionsPage />,
  },
  { path: '/profile', label: 'Profile', icon: User, element: <ProfilePage /> },
  { path: '/settings', label: 'Settings', icon: Settings, element: <SettingsPage />, footer: true },
];

export const primarySidebarRoutes = appRoutes.filter((route) => route.sidebar !== false && !route.footer);
export const footerSidebarRoutes = appRoutes.filter((route) => route.sidebar !== false && route.footer);
```

[src/pages/AddProblemPage.tsx \(lines 1-70 of 252\)](#)

```
import { Check, Sparkles } from 'lucide-react';
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
```

```

import { FormulaToolbar } from '../components/intake/FormulaToolbar';
import { UploadDropzone } from '../components/intake/UploadDropzone';
import { PageHeader } from '../components/layout/PageHeader';
import { FormulaBlock } from '../components/math/FormulaBlock';
import { PicoNote } from '../components/pico/PicoNote';
import { Button } from '../components/ui/Button';
import { Card } from '../components/ui/Card';
import { Tabs, type TabItem } from '../components/ui/Tabs';
import { sampleProblem, topicChips } from '../data/mockProblem';
import { picolabApi } from '../services/picolabApi';
import { writeCurrentParsedProblem, writeInputSource } from '../services/problemSession';

type AddProblemTab = 'scan' | 'type' | 'formula';

const tabs: Array<TabItem<AddProblemTab>> = [
  { value: 'scan', label: 'Scan image' },
  { value: 'type', label: 'Type problem' },
  { value: 'formula', label: 'Formula editor' },
];

export function AddProblemPage() {
  const navigate = useNavigate();
  const [tab, setTab] = useState<AddProblemTab>('scan');
  const [uploaded, setUploaded] = useState(false);
  const [problemText, setProblemText] = useState('');
  const [selectedChips, setSelectedChips] = useState<string[]>([]);
  const [formulaText, setFormulaText] = useState('v = v■ + at');
  const [pendingAction, setPendingAction] = useState<'reading' | 'scanning' | null>(null);

  const toggleChip = (chip: string) => {
    setSelectedChips((current) =>
      current.includes(chip) ? current.filter((item) => item !== chip) : [...current, chip],
    );
  };

  const goToScanConfirm = () => navigate('/scan-confirm');

  const analyzeProblem = async () => {
    if (pendingAction) return;

    setPendingAction('reading');
    writeInputSource('typed');

    try {
      const text = problemText.trim() || sampleProblem.text;
      const result = await picolabApi.parseProblem({
        mode: 'typed',
        text,
        subjectHint: selectedChips.some((chip) => chip === 'Algebra steps' || chip === 'Linear functions')
          ? 'math'
          : 'physics',
      });

      if (result.ok) {
        writeCurrentParsedProblem(result.data);
      }

      goToScanConfirm();
    } catch {
      goToScanConfirm();
    } finally {
      setPendingAction(null);
    }
  };

  const analyzeFormula = async () => {
    if (pendingAction) return;

```

src/pages/ScanConfirmPage.tsx (lines 1-70 of 365)

```

import { Check, Edit3, RefreshCw, RotateCw, Save, Search, X, ZoomIn, ZoomOut } from 'lucide-react';
import { useEffect, useMemo, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { HighlightLegend } from '../components/intake/HighlightLegend';
import { ScanPreview } from '../components/intake/ScanPreview';
import { PageHeader } from '../components/layout/PageHeader';
import { FormulaBlock } from '../components/math/FormulaBlock';
import { PicoMascot } from '../components/pico/PicoMascot';

```

```

import { Badge } from '../components/ui/Badge';
import { Button } from '../components/ui/Button';
import { Card } from '../components/ui/Card';
import { sampleProblem } from '../data/mockProblem';
import { picolabApi } from '../services/picolabApi';
import {
  readCurrentParsedProblem,
  readInputSource,
  writeCurrentParsedProblem,
  writeCurrentProblem,
  type InputSource,
} from '../services/problemSession';
import type { ParsedProblem } from '../types/api';

type EditableDetail = { value: string; description: string };

export function ScanConfirmPage() {
  const navigate = useNavigate();
  const [problemText, setProblemText] = useState(sampleProblem.text);
  const [parsedProblem, setParsedProblem] = useState<ParsedProblem | null>(null);
  const [inputSource, setInputSource] = useState<InputSource | null>(null);
  const [editMode, setEditMode] = useState(false);
  const [localDetails, setLocalDetails] = useState<EditableDetail[]>(sampleProblem.detectedValues);
  const [recheckState, setRecheckState] = useState(false);
  const [confirming, setConfirming] = useState(false);

  useEffect(() => {
    const storedProblem = readCurrentParsedProblem();
    const source = readInputSource();
    setInputSource(source);

    if (storedProblem) {
      setParsedProblem(storedProblem);
      setProblemText(storedProblem.statement);

      const details = storedProblem.extractedDetails.length
        ? storedProblem.extractedDetails.map((detail) => ({
            value: detail.unit ? `${detail.value} ${detail.unit}` : detail.value,
            description: detail.label,
          }))
        : sampleProblem.detectedValues;
      setLocalDetails(details);
    }
  }, []);

  const detectedDetails = useMemo(() => {
    if (!parsedProblem?.extractedDetails.length) {
      return sampleProblem.detectedValues;
    }
    return parsedProblem.extractedDetails.map((detail) => ({
      value: detail.unit ? `${detail.value} ${detail.unit}` : detail.value,
      description: detail.label,
    }));
  }, [parsedProblem]);

  const primaryAmbiguity = parsedProblem?.ambiguities[0];

  const isTypedInput = inputSource === 'typed' || inputSource === 'formula';
  const isScanInput = inputSource === 'scan';

  const leftPanelTitle = isTypedInput
    ? inputSource === 'formula'

```

src/pages/SmartNotebookPage.tsx (lines 1-80 of 646)

```

import {
  ArrowRight,
  Check,
  FlaskConical,
  Home,
  Lightbulb,
  Map,
  Plus,
  Route,
  RotateCcw,
  Sparkles,
  Target,
} from 'lucide-react';

```

```

import { useEffect, useMemo, useRef, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { NotebookStepCard, type StepAction, type StepPhase } from '../components/notebook/NotebookStepCard';
import { PicoCoachPanel } from '../components/notebook/PicoCoachPanel';
import { ProblemContextPanel } from '../components/notebook/ProblemContextPanel';
import { AskPicoDrawer } from '../components/pico/AskPicoDrawer';
import { Badge } from '../components/ui/Badge';
import { Button } from '../components/ui/Button';
import { Card } from '../components/ui/Card';
import { ProgressBar } from '../components/ui/ProgressBar';
import {
  mockNotebookProblem,
  mockPatternInsight,
  mockPicoCoach,
  notebookFlow,
  step1FormulaSignal,
  type LearningSignal,
  type LearningSignalSection,
  type NotebookProblem,
} from '../data/mockNotebook';
import { picolabApi } from '../services/picolabApi';
import { readCurrentProblem } from '../services/problemSession';
import {
  storeVisualLabSuggestionFromSignal,
  storeVisualLabSuggestionFromSignals,
} from '../services/visualLabSuggestion';
import type { ProblemEntity, StepCheckResponse } from '../types/api';

type StepNumber = 1 | 2 | 3 | 4;

const createProblemFromEntity = (problem: ProblemEntity | null): NotebookProblem => {
  if (!problem) return mockNotebookProblem;

  const knownValues = problem.extractedDetails
    .filter((detail) => detail.kind === 'given')
    .slice(0, 4)
    .map((detail) => ({
      label: detail.unit ? `${detail.value} ${detail.unit}` : detail.value,
      description: detail.label,
    }));

  return {
    ...mockNotebookProblem,
    eyebrow: `${problem.subject === 'physics' ? 'Physics' : 'Math'} · ${problem.topic}`,
    statement: problem.statement,
    knownValues: knownValues.length ? knownValues : mockNotebookProblem.knownValues,
    suggestedFormula: problem.suggestedFormula ?? mockNotebookProblem.suggestedFormula,
  };
};

// Deterministic Step 1 check: does the setup connect the right quantities?
const normalizeFormula = (value: string) =>
  value
    .toLowerCase()
    .replace(/\s+/g, '')
    .replace(/■/g, '0')
    .replace(/_/g, '')
    .replace(/[\.*]/g, '*');

const isValidStep1Setup = (value: string) => {
  const normalized = normalizeFormula(value);
  return normalized.includes('v0+at') || normalized.includes('2*5');
};

export function SmartNotebookPage() {
  const navigate = useNavigate();
  const [askPicoOpen, setAskPicoOpen] = useState(false);

```

src/pages/VisualLabPage.tsx (lines 1-60 of 687)

```

import { ArrowLeft, Bookmark, Target } from 'lucide-react';
import { useEffect, useMemo, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { FormulaBlock } from '../components/math/FormulaBlock';
import { LiveFormulaCard } from '../components/visual-lab/LiveFormulaCard';
import { MotionControls } from '../components/visual-lab/MotionControls';
import { MotionSimulation } from '../components/visual-lab/MotionSimulation';
import { VelocityTimeGraph } from '../components/visual-lab/VelocityTimeGraph';

```

```

import { VisualLabCoachPanel } from '../components/visual-lab/VisualLabCoachPanel';
import { VisualModeSelector } from '../components/visual-lab/VisualModeSelector';
import { AskPicoDrawer } from '../components/pico/AskPicoDrawer';
import { Badge } from '../components/ui/Badge';
import { Button } from '../components/ui/Button';
import { Card } from '../components/ui/Card';
import { visualLabCopy, visualLabDefaults, visualModes } from '../data/mockVisualLab';
import {
  clearVisualLabSuggestion,
  getTemplateDescription,
  getTemplateLabel,
  isVisualTemplateId,
  loadVisualLabSuggestion,
} from '../services/visualLabSuggestion';
import type { VisualTemplateId } from '../types/api';

type ActiveVisualMode = Extract<
  VisualTemplateId,
  'motion' | 'units' | 'graph' | 'formula' | 'free-body' | 'function'
>;

type TemplateInfo = {
  title: string;
  subtitle: string;
  detailTitle: string;
  detailItems: string[];
  badge: string;
  notice: string;
  why: string;
  picoNote: string;
  practiceLabel: string;
};

const supportedModes: ActiveVisualMode[] = [
  'motion',
  'units',
  'graph',
  'formula',
  'free-body',
  'function',
];

const isSupportedVisualMode = (mode?: string): mode is ActiveVisualMode =>
  isVisualTemplateId(mode) && supportedModes.includes(mode as ActiveVisualMode);

const toActiveMode = (mode?: string): ActiveVisualMode =>
  isSupportedVisualMode(mode) ? mode : 'motion';

const templateInfo: Record<ActiveVisualMode, TemplateInfo> = {
  motion: {
    title: 'Motion model',
    subtitle: 'Change time and acceleration to see how velocity grows.',
  },

```

src/pages/PracticeMissionsPage.tsx (lines 1-80 of 561)

```

import { useEffect, useMemo, useRef, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { DailyChallengeCard } from '../components/practice/DailyChallengeCard';
import { FocusMissionCard } from '../components/practice/FocusMissionCard';
import { MissionCompleteCard } from '../components/practice/MissionCompleteCard';
import { PracticeCoachPanel } from '../components/practice/PracticeCoachPanel';
import { RandomMissionPanel } from '../components/practice/RandomMissionPanel';
import { PageHeader } from '../components/layout/PageHeader';
import { AskPicoDrawer } from '../components/pico/AskPicoDrawer';
import { Card } from '../components/ui/Card';
import { dailyChallenge, focusMission, randomMissions } from '../data/mockMissions';
import { picolabApi } from '../services/picolabApi';
import {
  applyPracticeMissionCompletion,
  loadPracticeProgress,
  type PracticeProgress,
} from '../services/practiceProgress';
import { storeVisualLabSuggestionFromSignals } from '../services/visualLabSuggestion';
import type {
  PracticeAnswerResponse,
  PracticeCompleteResponse,
  PracticeMission,
} from '../types/api';

```

```

import type { Mission, PracticeQuestion } from '../types/mission';

const localDailyPractice: PracticeMission = {
  id: dailyChallenge.mission.id,
  title: dailyChallenge.mission.title,
  subject: 'physics',
  topic: dailyChallenge.mission.topic,
  difficulty: 'medium',
  prompt: dailyChallenge.problem,
  rewardPicoPoints: dailyChallenge.mission.reward.points,
};

const localFocusPractice: PracticeMission = {
  id: focusMission.id,
  title: focusMission.title,
  subject: 'physics',
  topic: 'Kinematics',
  difficulty: 'medium',
  prompt: focusMission.question.prompt,
  options: focusMission.question.options,
  rewardPicoPoints: 25,
};

const localRandomPractice: PracticeMission[] = randomMissions.map((mission) => ({
  id: mission.id,
  title: mission.title,
  subject: mission.subject === 'Math' ? 'math' : 'physics',
  topic: mission.topic,
  difficulty: mission.difficulty.toLowerCase() as PracticeMission['difficulty'],
  prompt: mission.title,
  rewardPicoPoints: mission.reward.points,
})));

const randomMissionDescriptions: Record<string, string | undefined> = Object.fromEntries(
  randomMissions.map((mission) => [mission.id, mission.description]),
);

const randomMissionQuestions: Record<string, Mission['question']> = Object.fromEntries(
  randomMissions.map((mission) => [mission.id, mission.question]),
);

const toMissionCard = (mission: PracticeMission): Mission => ({
  id: mission.id,
  title: mission.title,
  subject: mission.subject === 'math' ? 'Math' : 'Physics',
  topic: mission.topic,
  difficulty:
    mission.difficulty === 'easy' ? 'Easy' : mission.difficulty === 'hard' ? 'Hard' : 'Medium',
  reward: {
    label: `+${mission.rewardPicoPoints} PicoPoints`,
    points: mission.rewardPicoPoints,
  },
  description: randomMissionDescriptions[mission.id],
  question: randomMissionQuestions[mission.id],
});

const toPracticeQuestion = (mission: PracticeMission): PracticeQuestion => ({

```

src/pages/GrowthMapPage.tsx (lines 1-55 of 241)

```

import { MessageCircle } from 'lucide-react';
import { useEffect, useMemo, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { GrowthSummaryCard } from '../components/growth/GrowthSummaryCard';
import { LearningSignalRow } from '../components/growth/LearningSignalRow';
import { SuggestedDirectionCard } from '../components/growth/SuggestedDirectionCard';
import { PageHeader } from '../components/layout/PageHeader';
import { AskPicoDrawer } from '../components/pico/AskPicoDrawer';
import { PicoMascot } from '../components/pico/PicoMascot';
import { Button } from '../components/ui/Button';
import { Card } from '../components/ui/Card';
import { Tabs, type TabItem } from '../components/ui/Tabs';
import {
  growthFilters,
  growthLearningSignals,
  growthMapPicoInsight,
  growthMapSuggestedDirection,
  growthMapSummaryCards,

```



```

    type GrowthSignal,
    type GrowthSummaryCardData,
    type GrowthFilter,
  } from '../data/mockGrowth';
import {
  getPrioritizedDiagnosticSignals,
  getSuggestedGrowthFocusFromSignals,
  mergeGrowthMapWithLocalProgress,
} from '../services/learningProgress';
import { picolabApi } from '../services/picolabApi';
import { storeVisualLabSuggestionFromSignals } from '../services/visualLabSuggestion';
import type { GrowthMapResponse, LearningSignal } from '../types/api';

const filterTabs: Array<TabItem<GrowthFilter>> = growthFilters.map((filter) => ({
  label: filter.label,
  value: filter.value,
})));

const localGrowthMap: GrowthMapResponse = {
  mainFocus: growthMapSummaryCards[0].title,
  strongestSkill: growthMapSummaryCards[1].title,
  nextOpportunity: growthMapSummaryCards[2].title,
  learningSignals: growthLearningSignals.map((signal) => ({
    id: signal.id,
    kind:
      signal.id.startsWith('algebra.')
        ? 'signSlip'
        : signal.id.startsWith('concept.')
          ? 'quantityConfusion'
          : 'unitMismatch',
    title: signal.title,
    description: signal.description,
    strength: signal.strength,
    suggestedFocus: signal.bestNextAction.label,
  })),
  picoInsight: growthMapPicoInsight,
};

```

src/pages/GrowthPathPage.tsx (lines 1-55 of 284)

```

import { FlaskConical, Map, Target } from 'lucide-react';
import { useEffect, useMemo, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { CurrentGoalCard } from '../components/growth/CurrentGoalCard';
import { GrowthPathStep } from '../components/growth/GrowthPathStep';
import { RoadmapProgressCard } from '../components/growth/RoadmapProgressCard';
import { PageHeader } from '../components/layout/PageHeader';
import { AskPicoButton } from '../components/pico/AskPicoButton';
import { AskPicoDrawer } from '../components/pico/AskPicoDrawer';
import { PicoMascot } from '../components/pico/PicoMascot';
import { Badge } from '../components/ui/Badge';
import { Button } from '../components/ui/Button';
import { Card } from '../components/ui/Card';
import {
  currentGrowthGoal,
  growthPathPicoPlan,
  growthPathProgress,
  growthPathSteps,
  type GrowthPathStepData,
  recommendedGrowthStep,
} from '../data/mockGrowth';
import {
  getPrioritizedDiagnosticSignals,
  getSuggestedGrowthFocusFromSignals,
  readLearningProgress,
} from '../services/learningProgress';
import { picolabApi } from '../services/picolabApi';
import { storeVisualLabSuggestionFromSignals } from '../services/visualLabSuggestion';
import type { GrowthPathResponse } from '../types/api';

const localGrowthPath: GrowthPathResponse = {
  currentGoal: currentGrowthGoal.value,
  progressPercent: growthPathProgress.percent,
  recommendedFocus: recommendedGrowthStep.skill,
  steps: growthPathSteps.map((step) => ({
    id: step.id,
    title: step.title,
    status:

```

```

        step.status === 'recommended' ? 'recommended' : step.status === 'up-next' ? 'upNext' : 'later',
        reason: step.reason,
        items: step.items,
        route: step.route,
    })),
    picoPlan: growthPathPicoPlan,
};

const toStepData = (step: GrowthPathResponse['steps'][number], index: number): GrowthPathStepData => {
    const status =
        step.status === 'recommended'
            ? 'recommended'
            : step.status === 'upNext'
            ? 'up-next'
            : 'later';

    return {

```

src/components/practice/DailyChallengeCard.tsx (lines 1-70 of 203)

```

import { Check, Play, Route, Send } from 'lucide-react';
import { dailyChallenge } from '../../data/mockMissions';
import type { Mission, PracticeQuestion } from '../../types/mission';
import { FormulaBlock } from '../../math/FormulaBlock';
import { Badge } from '../../ui/Badge';
import { Button } from '../../ui/Button';
import { Card } from '../../ui/Card';
import { AnswerOptionButton } from './AnswerOptionButton';
import { MissionRewardBadge } from './MissionRewardBadge';
import { StreakRow } from './StreakRow';

type DailyChallengeCardProps = {
    onStart: () => void;
    mission?: Mission;
    title?: string;
    description?: string;
    problem?: string;
    formulaHint?: string;
    streakLabel?: string;
    streakDays?: string[];
    completed?: boolean;
    // Interactive challenge state (owned by the page).
    started?: boolean;
    question?: PracticeQuestion;
    selectedOptionId?: string | null;
    checked?: boolean;
    answerCorrect?: boolean;
    feedbackTitle?: string;
    feedbackBody?: string;
    completing?: boolean;
    earnedPoints?: number;
    onSelect?: (optionId: string) => void;
    onCheck?: () => void;
    onComplete?: () => void;
};

export function DailyChallengeCard({
    onStart,
    mission = dailyChallenge.mission,
    title = dailyChallenge.title,
    description = dailyChallenge.description,
    problem = dailyChallenge.problem,
    formulaHint = dailyChallenge.formulaHint,
    streakLabel = dailyChallenge.streakLabel,
    streakDays = dailyChallenge.streakDays,
    completed = false,
    started = false,
    question = dailyChallenge.question,
    selectedOptionId = null,
    checked = false,
    answerCorrect = false,
    feedbackTitle,
    feedbackBody,
    completing = false,
    earnedPoints = mission.reward.points,
    onSelect,
    onCheck,
    onComplete,

```

```

}: DailyChallengeCardProps) {
  const isInteractive = started && !completed;

  return (
    <Card className="overflow-hidden px-5 py-5">
      <div className="grid gap-5 lg:grid-cols-[minmax(0,1fr)_240px]">
        <div>
          <div className="mb-3 flex flex-wrap items-center gap-2">
            <Badge variant="blue">{mission.subject}</Badge>
            <Badge variant="grey">{mission.topic}</Badge>
            <Badge variant="yellow">{mission.difficulty}</Badge>
            <MissionRewardBadge reward={mission.reward} />
          </div>

```

src/components/practice/RandomMissionPanel.tsx (lines 1-70 of 164)

```

import { Check, Send, Shuffle } from 'lucide-react';
import type { Mission, MissionDifficulty } from '../../types/mission';
import { Badge, type BadgeVariant } from '../../ui/Badge';
import { Button } from '../../ui/Button';
import { Card } from '../../ui/Card';
import { AnswerOptionButton } from './AnswerOptionButton';
import { MissionRewardBadge } from './MissionRewardBadge';

type RandomMissionPanelProps = {
  mission: Mission;
  selectedOptionId: string | null;
  checked: boolean;
  completed: boolean;
  completing: boolean;
  onSelect: (optionId: string) => void;
  onCheck: () => void;
  onComplete: () => void;
  onTryAnother: () => void;
};

const difficultyVariant: Record<MissionDifficulty, BadgeVariant> = {
  Easy: 'green',
  Medium: 'yellow',
  Hard: 'coral',
};

export function RandomMissionPanel({
  mission,
  selectedOptionId,
  checked,
  completed,
  completing,
  onSelect,
  onCheck,
  onComplete,
  onTryAnother,
}: RandomMissionPanelProps) {
  const question = mission.question;
  const answerCorrect = Boolean(question && selectedOptionId === question.correctOptionId);
  const feedbackTitle = answerCorrect ? 'Nice work.' : 'Almost there.';
  const feedbackBody = question
    ? answerCorrect
      ? question.feedbackCorrect
      : question.feedbackUsefulSignal
    : '';

  return (
    <Card className="p-fade border-[#B8D8F4] px-5 py-5">
      <div className="flex items-start justify-between gap-3">
        <div>
          <div className="flex items-center gap-2">
            <Shuffle size={16} className="text-pico-blue" aria-hidden="true" />
            <h2 className="text-[17px] font-extrabold tracking-[-0.02em] text-pico-text">
              Random mission
            </h2>
          </div>
          <p className="mt-1 text-[13px] leading-relaxed text-pico-secondary">
            A short extra practice picked by Pico.
          </p>
        </div>
        {completed ? <Badge variant="green">Completed</Badge> : <Badge variant="blue">Optional</Badge>}
      </div>

```

```

<div className="mt-4 rounded-[14px] bg-pico-soft px-4 py-4">
  <h3 className="text-[15px] font-bold text-pico-text">{mission.title}</h3>
  <div className="mt-2 flex flex-wrap gap-1.5">
    <Badge variant="blue">{mission.subject}</Badge>
    <Badge variant="grey">{mission.topic}</Badge>
    <Badge variant={difficultyVariant[mission.difficulty]}>{mission.difficulty}</Badge>
    <MissionRewardBadge reward={mission.reward} />
  </div>
</div>

```

src/components/practice/MissionCompleteCard.tsx (lines 1-60 of 75)

```

import { CheckCircle2, Map, Plus, Route, Shuffle } from 'lucide-react';
import { missionComplete } from '../../data/mockMissions';
import { Button } from '../../ui/Button';
import { Card } from '../../ui/Card';

type MissionCompleteCardProps = {
  onViewRoadmap: () => void;
  onOpenGrowthMap: () => void;
  onTryRandomMission: () => void;
  onAddAnotherProblem: () => void;
  title?: string;
  copy?: string;
  stats?: Array<{
    label: string;
    value: string;
  }>;
  completing?: boolean;
};

export function MissionCompleteCard({
  onViewRoadmap,
  onOpenGrowthMap,
  onTryRandomMission,
  onAddAnotherProblem,
  title = missionComplete.title,
  copy = missionComplete.copy,
  stats = missionComplete.stats,
  completing = false,
}: MissionCompleteCardProps) {
  return (
    <Card variant="progress" className="p-fade px-5 py-5">
      <div className="flex items-start gap-3">
        <CheckCircle2 size={22} className="mt-0.5 shrink-0 text-[#2A7850]" aria-hidden="true" />
        <div>
          <h2 className="text-[17px] font-extrabold tracking-[-0.02em] text-[#2A7850]">
            {completing ? 'Saving mission progress' : title}
          </h2>
          <p className="mt-1 text-[13.5px] leading-relaxed text-[#2A7850]">
            {completing ? 'Pico is updating your streak and PicoPoints...' : copy}
          </p>
        </div>
      </div>

      <div className="mt-4 grid gap-2 sm:grid-cols-3">
        {stats.map((stat) => (
          <div key={stat.label} className="rounded-[10px] bg-white/70 px-3 py-2.5">
            <div className="text-[10.5px] font-bold uppercase tracking-[0.06em] text-[#2A7850]">
              {stat.label}
            </div>
            <div className="mt-1 text-[12.5px] font-semibold text-pico-text">{stat.value}</div>
          </div>
        ))}
      </div>

      <div className="mt-4 flex flex-wrap gap-2">
        <Button size="sm" onClick={onViewRoadmap}>
          <Route size={13} />
          View Roadmap
        </Button>
        <Button variant="secondary" size="sm" onClick={onOpenGrowthMap}>

```

src/services/apiClient.ts (81 lines)

```

import type { ApiError, ApiResult } from '../types/api';

const rawBaseUrl = import.meta.env?.VITE_PICOLAB_API_URL ?? 'http://127.0.0.1:8787/api';

export const API_BASE_URL = rawBaseUrl.replace(/\/$/, '');

```

```

type RequestOptions = {
  body?: unknown;
  headers?: Record<string, string>;
  timeoutMs?: number;
};

const createError = (code: string, message: string, details?: Record<string, unknown>): ApiError => ({
  code,
  message,
  details,
});

const request = async <T>({
  method: 'GET' | 'POST' | 'PATCH',
  path: string,
  options: RequestOptions = {},
}): Promise<ApiResult<T>> => {
  const controller = new AbortController();
  const timeoutId =
    options.timeoutMs && options.timeoutMs > 0
      ? globalThis.setTimeout(() => controller.abort(), options.timeoutMs)
      : undefined;

  try {
    const response = await fetch(`${API_BASE_URL}${path}`, {
      method,
      signal: controller.signal,
      headers: {
        'Content-Type': 'application/json',
        ...options.headers,
      },
      body: options.body === undefined ? undefined : JSON.stringify(options.body),
    });

    const payload = (await response.json()).catch(() => null) as T | ApiError | null;

    if (!response.ok) {
      return {
        ok: false,
        source: 'backend',
        error:
          payload && typeof payload === 'object' && 'code' in payload
            ? (payload as ApiError)
            : createError('backend_error', `Request failed with status ${response.status}`),
      };
    }

    return {
      ok: true,
      source: 'backend',
      data: payload as T,
    };
  } catch (error) {
    return {
      ok: false,
      source: 'backend',
      error: createError('network_unavailable', 'Backend is unavailable.', {
        cause: error instanceof Error ? error.message : String(error),
      }),
    };
  } finally {
    if (timeoutId) {
      globalThis.clearTimeout(timeoutId);
    }
  }
};

export const apiClient = {
  get: <T>(path: string, options?: Omit<RequestOptions, 'body'>) =>
    request<T>('GET', path, options),
  post: <T>(path: string, body?: unknown, options?: Omit<RequestOptions, 'body'>) =>
    request<T>('POST', path, { ...options, body }),
  patch: <T>(path: string, body?: unknown, options?: Omit<RequestOptions, 'body'>) =>
    request<T>('PATCH', path, { ...options, body }),
};

```

[src/services/picolabApi.ts \(lines 1-60 of 1198\)](#)

```

import { apiClient } from './apiClient';
import { mockPicolabApi } from './mockPicolabApi';
import type {
  ApiResult,
  AskPicoRequest,
  AskPicoResponse,
  ExtractedDetail,
  ExtractedDetailKind,
  GrowthMapResponse,
  GrowthPathRegenerateRequest,
  GrowthPathResponse,
  GrowthSignalRequest,
  LearningSignal,
  LearningSignalKind,
  ParsedProblem,
  PracticeAnswerRequest,
  PracticeAnswerResponse,
  PracticeCompleteRequest,
  PracticeCompleteResponse,
  PracticeMission,
  ProfileGoalsRequest,
  ProfileResponse,
  ProblemEntity,
  ProblemInput,
  ProblemScanInput,
  StepCheckRequest,
  VisualLabTemplateRequest,
  StepCheckResponse,
} from '../types/api';
import type { DiagnosticResult, LearningSignalInstance } from '../types/learningSignals';

const activeApi = mockPicolabApi;
const ASK_PICO_TIMEOUT_MS = 4000;
const CORE_FLOW_TIMEOUT_MS = 4000;
const PRACTICE_TIMEOUT_MS = 4000;
const GROWTH_TIMEOUT_MS = 4000;
const PROFILE_TIMEOUT_MS = 4000;

type BackendAskPicoEnvelope = {
  ok?: boolean;
  data?: {
    message?: {
      id?: unknown;
      role?: unknown;
      text?: unknown;
      content?: unknown;
    };
    suggestedQuestions?: unknown;
    suggestedActions?: unknown;
    actions?: unknown;
    learningSignal?: unknown;
  };
};

const isAction = (value: unknown): value is { label: string; route: string } =>
  Boolean(
    value &&
    typeof value === 'object' &&
    'label' in value &&
    'route' in value &&
  )

```

[src/services/diagnosticEngine.ts \(lines 1-80 of 324\)](#)

```

import {
  getSignalDefinition,
  getVisualTemplateForSignal,
  prioritizeSignals,
} from '../data/learningSignals';
import type {
  DiagnosticEvidence,
  DiagnosticResult,
  LearningSignalCategory,
  LearningSignalInstance,
  LearningSignalSeverity,
} from '../types/learningSignals';
import type { LearningSignal } from '../types/api';

const categoryBySignalId = (signalId: string): LearningSignalCategory =>

```

```

    signalId.split('.')[0] as LearningSignalCategory;

const subtypeBySignalId = (signalId: string) => signalId.split('.')[1] ?? signalId;

const normalize = (value?: string) => value?.trim().toLowerCase() ?? '';

const numberPattern = /-?\d+(?:\.\d+)?/;

const extractNumber = (value?: string) => {
    const match = value?.match(numberPattern);
    return match ? Number(match[0]) : undefined;
};

const extractUnit = (value?: string) => {
    const normalized = normalize(value).replace(/\\s+/g, '');
    const units = ['m/s^2', 'm/s^2', 'km/h', 'm/s', 'cm', 'kg', 'n', 'min', 's', 'm'];
    return units.find((unit) => normalized.includes(unit));
};

const hasNumber = (value?: string) => numberPattern.test(value ?? '');

const containsAny = (value: string, tokens: string[]) => tokens.some((token) => value.includes(token));

const makeInstance = (
    signalId: string,
    evidence: DiagnosticEvidence,
    severity: LearningSignalSeverity,
    confidence: number,
    evidenceText: string,
): LearningSignalInstance => {
    const definition = getSignalDefinition(signalId);

    return {
        id: `${signalId}:${evidence.source}:${evidence.problemId ?? evidence.missionId ?? 'demo'}:${evidence.stepId ?? 'step'}:${Math.round(confidence * 100)}\`,
        signalId,
        category: definition?.category ?? categoryBySignalId(signalId),
        severity,
        status: 'new',
        confidence,
        evidence: evidenceText,
        source: evidence.source,
        createdAt: new Date().toISOString(),
        relatedProblemId: evidence.problemId,
        relatedStepId: evidence.stepId,
        suggestedPractice: definition?.suggestedPractice,
        suggestedVisualTemplate: definition?.suggestedVisualTemplate,
    };
};

const dedupeSignals = (signals: LearningSignalInstance[]) => {
    const seen = new Set<string>();
    return signals.filter((signal) => {
        if (seen.has(signal.signalId)) return false;
        seen.add(signal.signalId);
        return true;
    });
};

const feedbackForSignal = (
    signal: LearningSignalInstance | undefined,
    evidence: DiagnosticEvidence,
) => {
    const definition = signal ? getSignalDefinition(signal.signalId) : undefined;
    const expectedUnit = evidence.expectedUnit;
    const studentUnit = extractUnit(evidence.studentAnswer ?? evidence.studentStep);

```

src/data/learningSignals.ts (lines 1-55 of 456)

```

import type {
    LearningSignalCategory,
    LearningSignalDefinition,
    LearningSignalInstance,
} from '../types/learningSignals';

export const learningSignalDefinitions: LearningSignalDefinition[] = [
    {
        id: 'algebra.sign_error',

```

```

category: 'algebra',
subtype: 'sign_error',
title: 'Sign adjustment',
studentFriendlyLabel: 'A sign needs attention',
description: 'The reasoning is close, and a positive or negative sign changed along the way.',
evidenceExamples: ['Student wrote -8 when the rearranged expression gives +8.'],
commonPatterns: ['A negative sign is dropped when moving a term.', 'Direction is not carried into the substitution.'],
supportiveFeedback: 'Your setup is useful. The adjustment is tracking the sign through each line.',
suggestedPractice: ['Sign check sprint', 'Equation balance steps'],
suggestedVisualTemplate: 'formula',
growthPathFocus: ['Algebra fluency', 'Direction and sign meaning'],
},
{
  id: 'algebra.isolation_error',
  category: 'algebra',
  subtype: 'isolation_error',
  title: 'Variable isolation',
  studentFriendlyLabel: 'The target variable needs a cleaner path',
  description: 'The student is working toward the right target but needs support isolating it.',
  evidenceExamples: ['Student leaves t on both sides after trying to solve for time.'],
  commonPatterns: ['Inverse operations happen out of sequence.', 'A coefficient remains attached to the target.'],
  supportiveFeedback: 'You identified the target. Now use one inverse operation at a time to isolate it.',
  suggestedPractice: ['Solve-for-target steps', 'One-operation isolation'],
  suggestedVisualTemplate: 'formula',
  growthPathFocus: ['Formula rearrangement', 'Algebra fluency'],
},
{
  id: 'algebra.order_of_operations',
  category: 'algebra',
  subtype: 'order_of_operations',
  title: 'Operation order',
  studentFriendlyLabel: 'The operation order needs attention',
  description: 'The student uses the right operations but applies them in a confusing order.',
  evidenceExamples: ['Student adds before multiplying inside a grouped expression.'],
  commonPatterns: ['Parentheses are skipped.', 'Multiplication and addition are swapped in sequence.'],
  supportiveFeedback: 'The operations are available. The useful signal is choosing the order that preserves the expression.',
  suggestedPractice: ['Order of operations warmup', 'Expression structure check'],
  suggestedVisualTemplate: 'formula',
  growthPathFocus: ['Algebra fluency'],
},
{
  id: 'algebra.substitution_structure',
  category: 'algebra',
  subtype: 'substitution_structure',
  title: 'Substitution structure',
  studentFriendlyLabel: 'The substitution structure needs attention',

```

server/src/index.ts (63 lines)

```

import 'dotenv/config';
import cors from 'cors';
import express from 'express';
import { askPicoRouter } from './routes/askPico.js';
import { growthRouter } from './routes/growth.js';
import { healthRouter } from './routes/health.js';
import { notebooksRouter } from './routes/notebooks.js';
import { practiceRouter } from './routes/practice.js';
import { problemsRouter } from './routes/problems.js';
import { profileRouter } from './routes/profile.js';
import { settingsRouter } from './routes/settings.js';
import { visualLabRouter } from './routes/visualLab.js';
import { errorHandler, notFoundHandler } from './middleware/errorHandler.js';

const app = express();
const port = Number(process.env.PORT ?? 8787);
const host = process.env.HOST ?? '127.0.0.1';
const defaultOrigins = ['http://localhost:5173', 'http://127.0.0.1:5173'];
const configuredOrigins = process.env.CLIENT_ORIGIN?.split(',')
  .map((origin) => origin.trim())
  .filter(Boolean);
const allowedOrigins = configuredOrigins?.length ? configuredOrigins : defaultOrigins;

app.use(
  cors({
    origin(origin, callback) {

```



```

    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true);
      return;
    }

    callback(new Error(`CORS origin not allowed: ${origin}`));
  },
  {}),
);
app.use(express.json({ limit: '1mb' }));

app.use('/api', healthRouter);
app.use('/api', askPicoRouter);
app.use('/api', problemsRouter);
app.use('/api', notebooksRouter);
app.use('/api', visualLabRouter);
app.use('/api', growthRouter);
app.use('/api', practiceRouter);
app.use('/api', profileRouter);
app.use('/api', settingsRouter);

app.use('/api', notFoundHandler);
app.use(errorHandler);

const server = app.listen(port, host, () => {
  console.log(`PicoLab mock API listening on http://${host}:${port}/api`);
});

const shutdown = (signal: NodeJS.Signals) => {
  console.log(`PicoLab mock API received ${signal}; shutting down.`);
  server.close(() => {
    process.exit(0);
  });
};

process.on('SIGINT', shutdown);
process.on('SIGTERM', shutdown);

```

server/src/routes/notebooks.ts (lines 1-80 of 96)

```

import { Router } from 'express';
import { notebook } from '../data/mockResponses.js';
import { createResponseMeta } from '../middleware/errorHandler.js';
import { learningSignalFromInstance, runMockDiagnostic } from '../services/diagnosticEngine.js';

export const notebooksRouter = Router();

notebooksRouter.get('/notebooks/:problemId', (req, res) => {
  res.json({
    ok: true,
    data: {
      ...notebook,
      problemSummary: {
        ...notebook.problemSummary,
        id: req.params.problemId,
      },
      source: 'mock',
    },
    meta: createResponseMeta(),
  });
});

notebooksRouter.post('/notebooks/:problemId/check-step', (req, res) => {
  const stepText = typeof req.body?.stepText === 'string' ? req.body.stepText : '';
  const expectedUnit = typeof req.body?.expectedUnit === 'string' ? req.body.expectedUnit : 'm/s';
  const expectedQuantity =
    typeof req.body?.expectedQuantity === 'string' ? req.body.expectedQuantity : 'final velocity';
  const diagnostic = runMockDiagnostic({
    source: 'notebook',
    studentStep: stepText,
    studentAnswer: stepText,
    expectedAnswer: typeof req.body?.expectedAnswer === 'string' ? req.body.expectedAnswer : '10 m/s',
    expectedUnit,
    expectedQuantity,
    expectedFormula: typeof req.body?.expectedFormula === 'string' ? req.body.expectedFormula : undefined,
    knownValues: Array.isArray(req.body?.knownValues) ? req.body.knownValues : undefined,
    target: 'velocity',
    topic: 'Kinematics',
  });
});

```

```

    problemText: 'An object starts at 2 m/s and accelerates at 4 m/s2 for 2 seconds.',
    problemId: req.params.problemId,
    stepId: typeof req.body?.stepId === 'string' ? req.body.stepId : 'step-2',
  });
  const learningSignal = learningSignalFromInstance(diagnostic.primarySignal);
  const includesVelocityUnit = diagnostic.signals.length === 0 || /m\s\b/.test(stepText);

  res.json({
    ok: true,
    data: {
      status: includesVelocityUnit ? 'strong' : 'needsAttention',
      supportiveFeedback: includesVelocityUnit
        ? 'Nice adjustment. The final unit now matches velocity.'
        : diagnostic.supportiveFeedback,
      whatWentWell: diagnostic.whatWentWell,
      whatToAdjust: includesVelocityUnit
        ? 'Add a brief interpretation of what the final velocity means.'
        : diagnostic.whatToAdjust,
      whyItMatters: diagnostic.whyItMatters,
      learningSignal: includesVelocityUnit ? undefined : learningSignal,
      primarySignal: diagnostic.primarySignal,
      signals: diagnostic.signals,
      diagnostic,
      suggestedNextAction: includesVelocityUnit
        ? 'Interpret the result.'
        : diagnostic.suggestedVisualTemplate
          ? 'Open the visual support.'
          : 'Try the suggested practice.',
      source: 'mock',
    },
    meta: createResponseMeta(),
  });
});

notebooksRouter.post('/notebooks/:problemId/hint', (_req, res) => {
  res.json({
    ok: true,
    data: {
      hint: 'Multiply the units first: m/s2 times s leaves m/s.',
      suggestedAction: { label: 'Open Visual Lab', route: '/visual-lab' },
      source: 'mock',
    },
  },

```

server/src/services/diagnosticEngine.ts (lines 1-60 of 324)

```

import {
  getSignalDefinition,
  getVisualTemplateForSignal,
  prioritizeSignals,
} from '../data/learningSignals.js';
import type {
  DiagnosticEvidence,
  DiagnosticResult,
  LearningSignalCategory,
  LearningSignalInstance,
  LearningSignalSeverity,
} from '../types/learningSignals.js';
import type { LearningSignal } from '../types/api.js';

const categoryBySignalId = (signalId: string): LearningSignalCategory =>
  signalId.split('.')[0] as LearningSignalCategory;

const subtypeBySignalId = (signalId: string) => signalId.split('.')[1] ?? signalId;

const normalize = (value?: string) => value?.trim().toLowerCase() ?? '';

const numberPattern = /-?\d+(?:\.\d+)?/;

const extractNumber = (value?: string) => {
  const match = value?.match(numberPattern);
  return match ? Number(match[0]) : undefined;
};

const extractUnit = (value?: string) => {
  const normalized = normalize(value).replace(/\s+/g, '');
  const units = ['m/s^2', 'm/s^2', 'km/h', 'm/s', 'cm', 'kg', 'n', 'min', 's', 'm'];
  return units.find((unit) => normalized.includes(unit));
};

```

```

const hasNumber = (value?: string) => numberPattern.test(value ?? '');

const containsAny = (value: string, tokens: string[]) => tokens.some((token) => value.includes(token));

const makeInstance = (
  signalId: string,
  evidence: DiagnosticEvidence,
  severity: LearningSignalSeverity,
  confidence: number,
  evidenceText: string,
): LearningSignalInstance => {
  const definition = getSignalDefinition(signalId);

  return {
    id: `${signalId}:${evidence.source}:${evidence.problemId ?? evidence.missionId ?? 'demo'}:${evidence.stepId ?? 'step'}:${Math.round(confidence * 100)}\``,
    signalId,
    category: definition?.category ?? categoryBySignalId(signalId),
    severity,
    status: 'new',
    confidence,
    evidence: evidenceText,
    source: evidence.source,
    createdAt: new Date().toISOString(),
    relatedProblemId: evidence.problemId,
    relatedStepId: evidence.stepId,
    suggestedPractice: definition?.suggestedPractice,
  };
}

```

Commit Summary

Recent history (git log --oneline --decorate -20):

```
937b034 (HEAD -> main, origin/main) fix: simplify random practice flow
2f886de fix: clarify practice completion actions
05f14ee fix: make practice daily challenge actionable
5c1df5c fix: clarify practice missions experience
c397eb2 fix: clarify notebook completion path
842aaad fix: make notebook solving flow interactive
39b791c fix: polish full student experience
c2af4dd fix: clarify student problem solving flow
c66259f fix: preserve visual lab suggestion from notebook
428533d chore: prepare canonical demo flow
318b126 feat: update branding and roadmap naming
7ba77df docs: prepare hackathon demo
3bea96b feat: activate visual lab templates
de6cc89 feat: enrich growth with diagnostic signals
70480c3 feat: add mock diagnostic engine
a7e4eaf feat: add learning signal taxonomy
2fe70f2 feat: sync growth and profile with mock backend
9fef30f feat: connect practice missions to mock backend
6a400bf feat: connect problem flow to mock backend
6922522 feat: connect ask pico to mock backend
```

Verification

Build / QA commands:

```
npm run build          # frontend (tsc -b && vite build)
npm run build:server   # backend  (tsc -p server/tsconfig.json)
git diff --check       # whitespace / conflict markers
```

The project builds successfully when these commands pass with no errors.